

Lab #1 EGRE246

This lab will consist of an introduction to test driven development (TDD) using the Unity unit test framework for C.

1. Download and install the Unity code in a directory where you will be developing your C programs.

Step 1 Download (from <http://www.throwtheswitch.org/unity>, at the bottom of the page – find “DOWNLOAD ZIP”) and unzip the file `Unity-master.zip` into your C source code directory. This should create a folder called ‘Unity-master’.

Step 2 Copy the files `unity.c`, `unity.h`, and `unity_internals.h` to the same directory/folder where you will be developing your C code. (If you are using onlineGDB you must load all files, including the header files (except the standard library ones), into onlineGDB.)

For this lab you are going to develop the C function `mypow` which returns its first argument raised to its second argument power, e.g. $\text{mypow}(2,3) \equiv 2^3 \equiv 8.0$, $\text{mypow}(-3,3) \equiv -3^3 \equiv -27.0$, $\text{mypow}(2,-2) \equiv 2^{-2} \equiv 0.25$, $\text{mypow}(0,-2) \equiv 0^{-2} \equiv \text{NaN}$, $\text{mypow}(14,0) \equiv 14^0 \equiv 1.0$.

2. Create the specification header and implementation files for your `mypow` function:

- file `mypow.h` containing the prototype:

```
float mypow(int, int);
```

- file `mypow.c` containing a syntactically-correct implementation stub:

```
#include "mypow.h"
float mypow(int x, int y) {
    return -9999; // value is arbitrary
}
```

3. Now begin to develop your Unity test code for your `mypow` function. **Do not use the builtin C function `pow` or equivalent** – write it yourself using a loop. Review the lecture notes to understand how TDD works.

- the beginnings of file `mypowUnity.c`:

```
#include <stdio.h>
#include "unity.h"
#include "mypow.h"

void setUp(void) {}
void tearDown(void) {}

// test suites will go here - see class notes

int main ( void ){
```

```

    UNITY_BEGIN();
    // will invoke test suites here - see class notes
    return UNITY_END();
}

```

- Compile your code with the command ‘gcc unity.c mypow.c mypowUnity.c’ (just click ‘Run’ with onlineGDB)
4. Now write and invoke some test suites/tests; here is an example to get you started (review the square root example in your notes for more helps):

```

void powtest1(void) {
    TEST_ASSERT_FLOAT_WITHIN_MESSAGE(0.01,8.0,mypow(2,3),"test 1-1");
    TEST_ASSERT_FLOAT_WITHIN_MESSAGE(0.01,-8.0,mypow(-2,3),"test 1-2");
    TEST_ASSERT_FLOAT_WITHIN_MESSAGE(0.01,234256.0,mypow(22,4),"test 1-3");
    TEST_ASSERT_FLOAT_WITHIN_MESSAGE(0.01,4782969.0,mypow(3,14),"test 1-4");
}

```

And add this to main: RUN_TEST(powtest1);

5. Edit your mypow function until it passes these tests.
6. Now add more tests and continue to modify your mypow function until all tests pass.